

The Linux System Call Execution Model: An Insight

This two-part series of articles focuses on the system call execution model in Linux based operating systems. This first part explains what system calls are, why they are required and the role of the glibc wrapper in system call execution. It then touches on the system call execution model from the Linux kernel perspective.



A system call is a request by a user space program (application) to interact with an OS executing in the kernel space. Basically, the user application invokes a system call when it requires access to the services that can only be accessed through a higher privilege mode — for example, creating a new task, doing network I/O or file I/O, or accessing hardware resources. These operations cannot be directly performed by the user space application; hence, operating systems like Linux provide a set of routines called system calls which are basically C functions executing in the kernel space.

When a user space program invokes a system call, there is a software interrupt (nowadays x86-64 provides *syscall* instruction for fast system call execution) and the mode switches from user space to kernel space (or more precisely, the privilege mode changes from lower to higher). Now the system call handler in the kernel space performs the required operation on behalf of the user space application and sends the response back to it.

We will see in detail in later sections as to how the user space to kernel space mode switching happens and how kernel space system call handlers are invoked. But first let's examine the role of the standard C library in the execution of system calls.

Note: x86-64 CPUs have a concept called privilege levels. A privilege level in the x86 instruction set controls the access of the program currently running on the processor to resources such as memory regions, I/O ports, and special instructions. There are four privilege levels ranging from 0, which is the most privileged, to 3 which is least privileged. Most modern operating systems use level 0 for the kernel/executive and use level 3 for application programs. Hence, the kernel runs at the most privileged level, called Ring 0, and user programs run at a lesser level, typically Ring 3.

Any resource available to level *n* is also available to levels 0 to *n*, so the privilege levels are rings. When a lesser privileged process tries to access a higher privileged process, a general protection fault exception is reported to the OS.